Learn Cilium

eBPF e Cilium sem mistério: do primeiro hook à carga do programa

Teresina

2026

Learn Cilium

eBPF e Cilium sem mistério: do primeiro hook à carga do programa

Edição 0.1 piloto. Público-alvo: iniciantes.

Teresina
2026

APRESENTAÇÃO

Este piloto acompanha um programa eBPF — um programa restrito que o kernel Linux pode carregar — do pedido de carga até um evento. O kernel é o núcleo do Linux; hook é um ponto previsto de ligação, e evento é a ocorrência que chega àquele contexto. Espaço de usuário é a área fora do kernel onde processos executam; ali, o agente do Cilium configura componentes, enquanto o programa ligado ao hook atua no contexto do kernel. O caminho exato depende do tipo de programa e da configuração.

Vocabulário de configuração: YAML é um formato textual para dados. No Kubernetes, label (rótulo) é um par chave/valor de metadados que ajuda a identificar e selecionar objetos, mas não autentica; no exemplo ilustrativo `app: api`, `app` é a chave e `api` é o valor (Kubernetes Authors, 2026). Um manifesto Kubernetes é um documento declarativo que descreve um recurso; policy (política de tráfego) é uma regra declarada para controlar comunicações (Cilium Authors, 2026). Nas citações autor-data, 2026 é o ano de consulta quando a página não informa o ano de publicação; `[s.d.]` nas referências conserva essa ausência sem inventar data.

As figuras são modelos didáticos originais. O fragmento YAML e os nomes `frontend`, `api` e `backend` são exemplos ilustrativos; não descrevem um cluster observado. A documentação stable do Cilium consultada em 28 de outubro de 2026 indicava 1.20.4; isso é referência documental, não versão instalada ou executada.

LISTA DE FIGURAS

Figura 1 – O agente configura; o programa atua no hook 7
Figura 2 – Sequência temporal: loader, kernel e hook 8
Figura 3 – Decisão do verifier e limite da conclusão 9

LISTA DE QUADROS

Quadro 1 – Vocabulário em uma frase 6

LISTA DE TABELAS

Tabela 1 – Verificações e limites da conclusão 8

SUMÁRIO

1	**UM PROGRAMA EBPF ENCONTRA SEU EVENTO**	**6**
1.1	**O que você vai aprender**	**6**
1.2	**Palavras antes do jargão**	**6**
1.3	**Um ponto de ligação, dois papéis**	**6**
1.4	**Da solicitação ao evento**	**7**
1.5	**Aceito ou rejeitado: o alcance do verifier**	**7**
1.6	**Pratique e confira**	**10**
	Referências	**11**

1 UM PROGRAMA EBPF ENCONTRA SEU EVENTO

1.1 O que você vai aprender

Ao final, você poderá:

- explicar com palavras simples o que é um hook e onde um programa eBPF se conecta;
- ordenar solicitação de carga, verificação, aceitação ou rejeição, attach e execução por evento;
- distinguir o que a aprovação do verifier indica daquilo que ela não prova, e separar o agente Cilium do datapath.

Pré-requisito: nenhum cluster ou terminal. Leia a sequência como um modelo conceitual; ela não é uma receita de instalação.

1.2 Palavras antes do jargão

Quadro 1 – Vocabulário em uma frase

Termo	**Definição curta e limite**
Kernel / eBPF	O kernel é o núcleo do Linux; eBPF carrega programas restritos que respondem a certos eventos.
Hook / evento	Hook é um ponto previsto de ligação; evento é a ocorrência que chega a esse contexto.
Verifier / mapa	Verifier analisa propriedades de segurança na carga; mapa guarda valores associados a chaves.
Attach / JIT	Attach liga um programa aceito ao hook; JIT compila para instruções nativas quando aplicável.
Agente / datapath	O agente Cilium configura componentes; o datapath processa tráfego. Cada um tem um papel diferente.

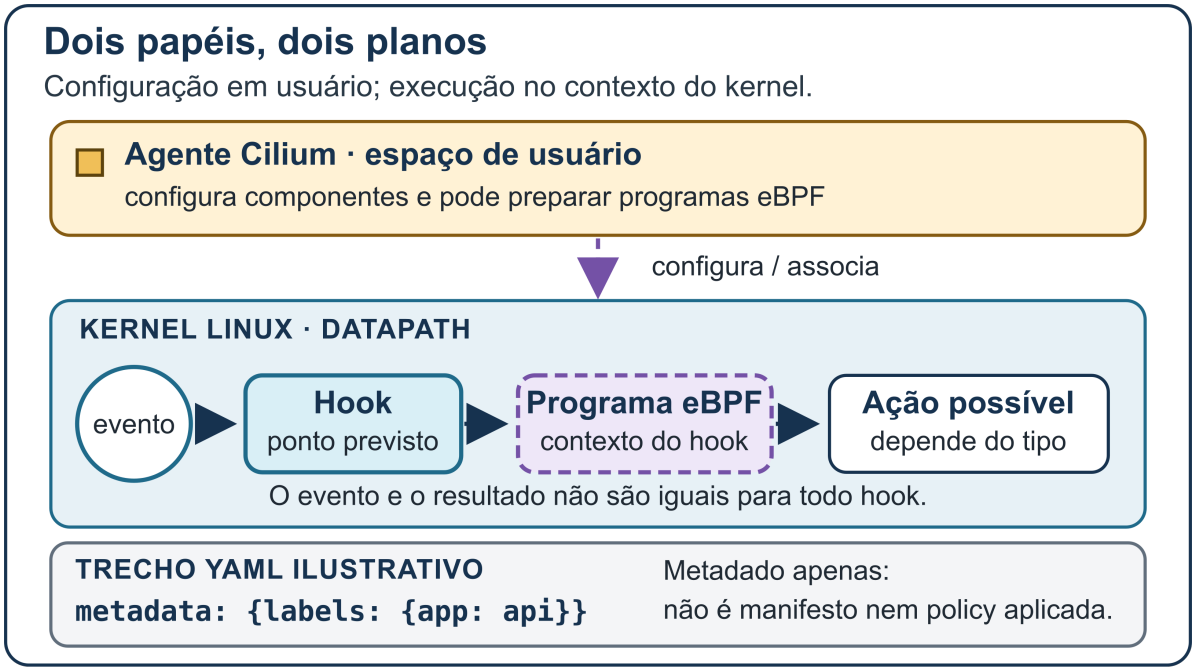
Fonte: Elaboração própria (2026), com base em (eBPF Foundation, 2026), (Linux Kernel Documentation, 2026) e (Cilium Authors, 2026).

1.3 Um ponto de ligação, dois papéis

A analogia do hook como ponto de conexão ajuda a localizar o attach; ele não é um guarda geral, e contexto e resultado variam por tipo.

Leitura da Figura 1: agente configura; no kernel, evento → hook → programa → ação por tipo. O fragmento YAML `metadata: {labels: {app: api}}` é ilustrativo, não manifesto nem policy aplicada; label não autentica. (Kubernetes Authors, 2026).

Figura 1 – O agente configura; o programa atua no hook



Fonte: Elaboração própria (2026), com base em (Cilium Authors, 2026), (eBPF Foundation, 2026) e (Kubernetes Authors, 2026).

1.4 Da solicitação ao evento

No modelo temporal, um loader em espaço de usuário solicita a carga ao kernel. O verifier do kernel analisa o programa; se rejeita, a tentativa termina. Se aceita, o kernel pode compilar com JIT quando aplicável. Então o loader solicita attach, o kernel associa o programa ao hook e um evento compatível chega ao hook, onde o programa executa no contexto do kernel. Esta é uma sequência didática, pois os mecanismos de carga e attach variam por tipo de programa. (eBPF Foundation, 2026; Linux Kernel Documentation, 2026; Cilium Authors, 2026)

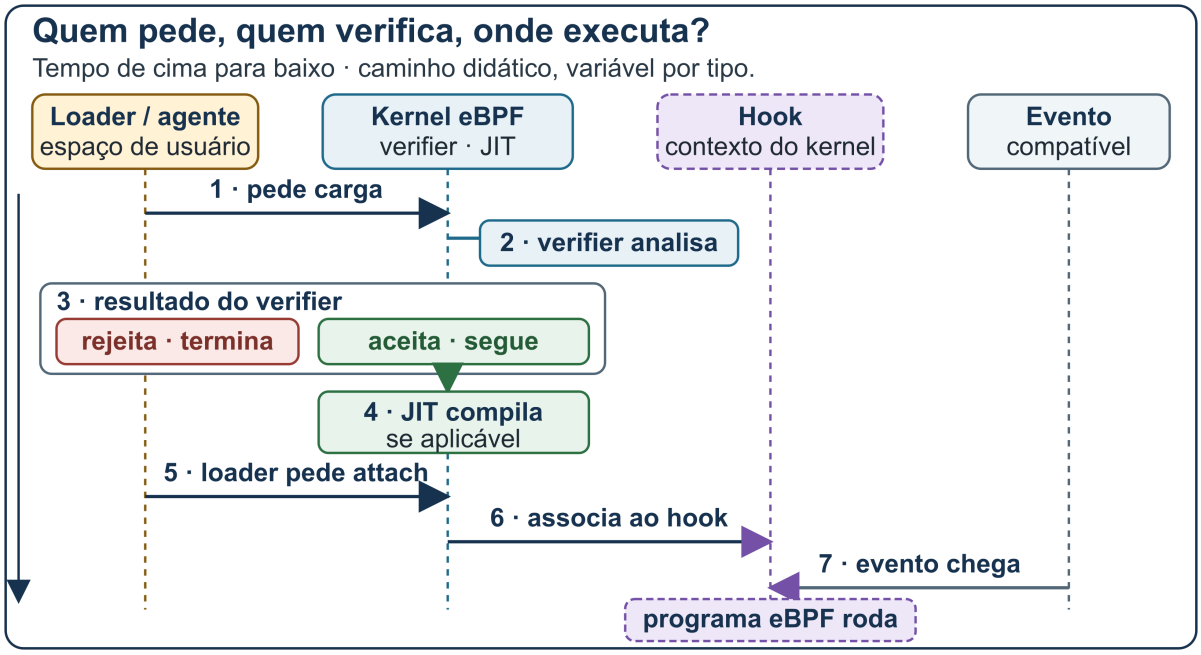
No caminho rejeitado, a tentativa termina; no caminho aceito, JIT permanece condicional e o loader pede attach ao kernel.

1.5 Aceito ou rejeitado: o alcance do verifier

O verifier analisa tipos, estado, acessos à memória e caminhos. Se não prova as condições aplicáveis, rejeita a carga. Se aceita, o programa passou nessas verificações; isso não confirma a intenção lógica. (Linux Kernel Documentation, 2026).

Leitura do fluxo: se as propriedades não são provadas, a carga é rejeitada; se são, o programa pode seguir para attach, sem validar a lógica desejada.

Figura 2 – Sequência temporal: loader, kernel e hook



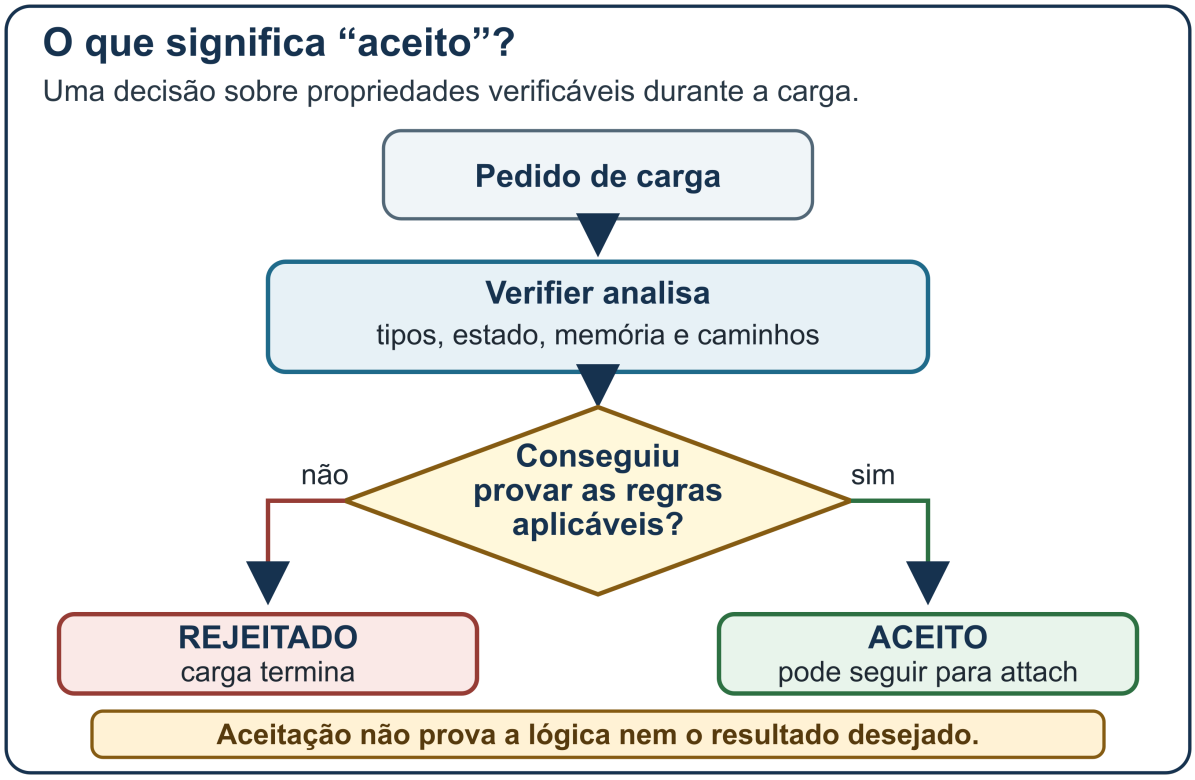
Fonte: Elaboração própria (2026), com base em (eBPF Foundation, 2026) e (Linux Kernel Documentation, 2026).

Tabela 1 – Verificações e limites da conclusão

O verifier analisa	**Isso ajuda a estabelecer**	**Continua sem prova**
Tipos e estado	uso compatível dos valores acompanhados	que a regra de negócio é a desejada
Acessos à memória	respeito aos limites verificados	que um serviço respondeu corretamente
Caminhos do programa	que os caminhos analisados atendem às restrições	que o tráfego terá o resultado esperado

Fonte: Elaboração própria (2026), com base em (Linux Kernel Documentation, 2026).

Figura 3 – Decisão do verifier e limite da conclusão



Fonte: Elaboração própria (2026), com base em (Linux Kernel Documentation, 2026).

1.6 Pratique e confira

Exercício conceitual (15 minutos). Ordene: (B) solicitar carga, (D) verifier, (F) aceitar/rejeitar, (A) attach, (C) evento e (E) execução. Identifique qual etapa aparece como condicional e em que ponto da sequência ocorre; não execute comandos.

Gabarito comentado: B → D → F; rejeitado, termina. Aceito: a etapa JIT aparece depois da aceitação e antes do attach, quando aplicável; então vêm attach, evento e execução (A → C → E). Isso não prova que a lógica é correta.

Perguntas para recordar e transferir: (1) Hook e evento são iguais? (2) O que o verifier analisa? (3) O que a aceitação não prova? (4) Como diferem agente e datapath?

Erro comum: "O verifier aceitou, então a policy está correta." A conclusão não decorre da verificação: uma análise de segurança de execução não valida, por si só, a intenção lógica ou o comportamento completo da aplicação.

Resumo: o agente configura componentes; o kernel verifica a carga; attach conecta programa e hook; um evento compatível pode executar o programa; aceitar não prova a lógica.

REFERÊNCIAS

Cilium Authors. Component overview e terminology (documentação stable 1.20.4). [s.d.]. Acesso em: 28 out. 2026. Disponível em: component overview e terminology.

eBPF Foundation. What is eBPF? [s.d.]. Acesso em: 28 out. 2026. Disponível em: ebpf.io/what-is-ebpf.

Kubernetes Authors. Labels and selectors. [s.d.]. Acesso em: 28 out. 2026. Disponível em: Kubernetes labels.

Linux Kernel Documentation. BPF verifier e BPF maps. [s.d.]. Acesso em: 28 out. 2026. Disponível em: BPF verifier e BPF maps.